Proyecto Integrador · Módulo 5 Backend · Henry

GitHub MCP Server

Automatizar GitHub con lenguaje natural

Romina Babiel

github.com/rominababiel/github-mcp-server

02

El problema

01

Los equipos repiten muchas veces por día las mismas tareas en GitHub: crear repos, abrir y cerrar issues, crear ramas, abrir pull requests.

02

Cada tarea obliga a salir del editor, entrar a la web o recordar comandos.

03

Un asistente que solo conversa explica cómo hacerlo, pero **no lo hace**.

03 La solución

Un **server MCP** que expone operaciones de GitHub como **tools**.

> “Crea una rama feat/docs en mi repo demo, agrega docs/guia.md y abre un pull request”

↓ El agente elige y ejecuta en orden:

`create_branch` → `create_commit` → `create_pull_request` → **devuelve el link al PR**

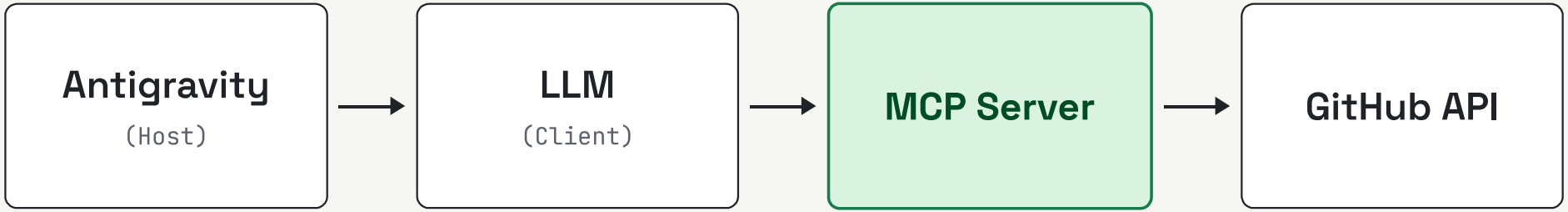
04 Chatbot, asistente y agente

	Qué hace
Chatbot	Responde con texto. No actúa
Asistente AI	Ayuda con una tarea a pedido, en una conversación
AI Agent	Decide qué herramientas usar y las ejecuta para cumplir un objetivo

Mi proyecto le da al agente las **herramientas** para actuar sobre GitHub.

05

Arquitectura



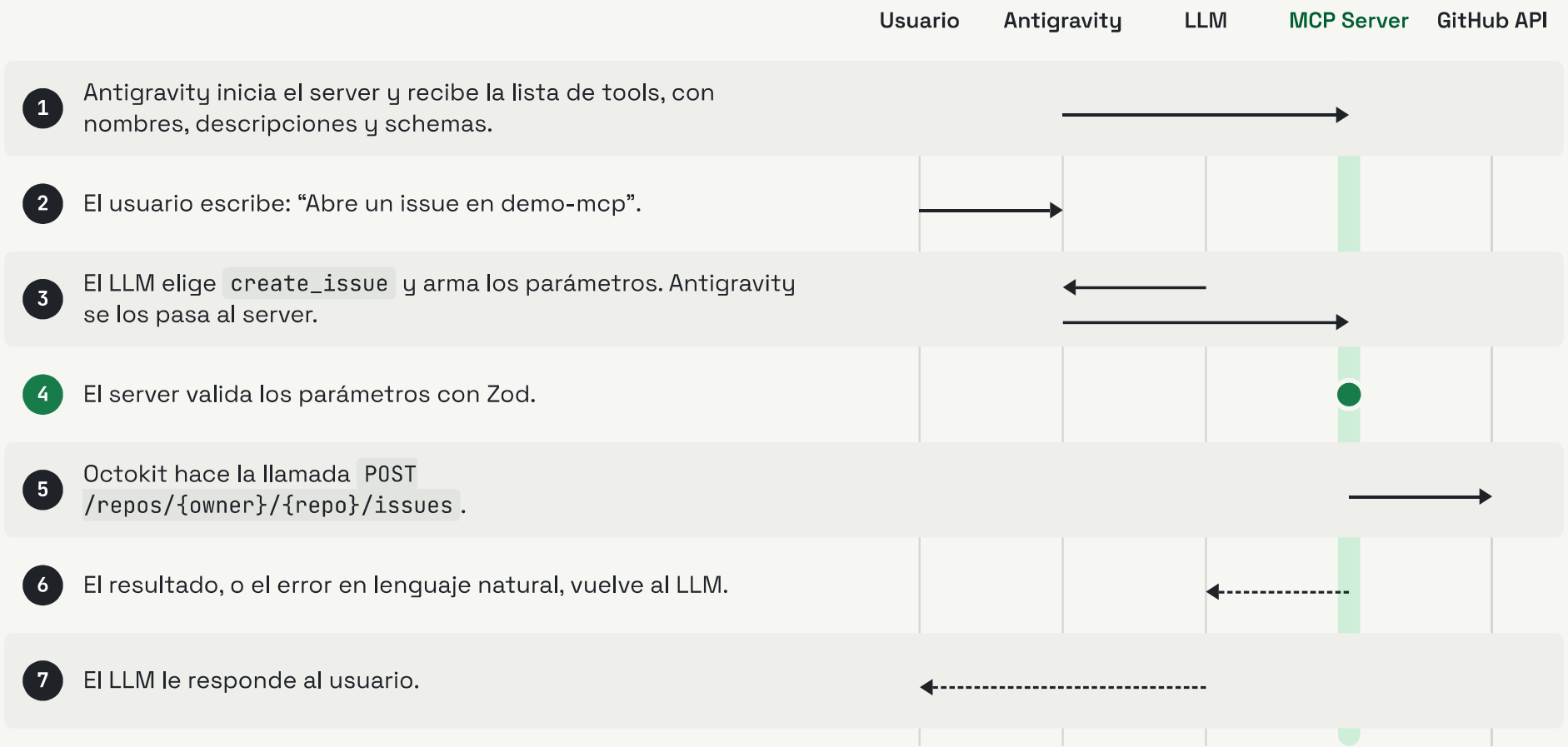
Antigravity lanza el server como proceso local y se comunica por **stdio con JSON-RPC**.

El LLM (Gemini 3.8 Flash) decide qué tool usar.

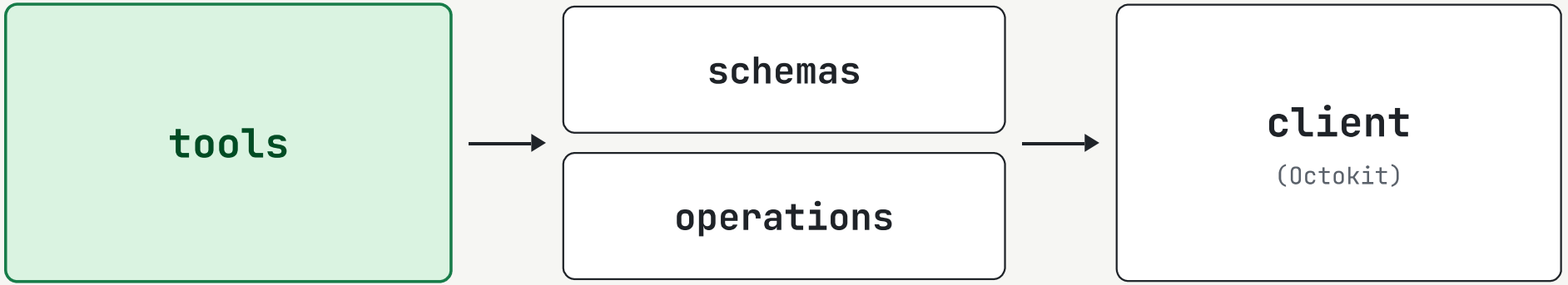
El server valida con **Zod** y llama a GitHub con **Octokit**.

06

Recorrido de un pedido



07 Capas del código



Además:

errors: tipos de error y traducción a lenguaje natural.

utils: logs y reintentos.

08 Stack

Node.js + TypeScript

`@modelcontextprotocol/sdk`
el SDK oficial de MCP

`@octokit/rest`
el cliente oficial de GitHub

Zod
validación y tipos

Vitest
tests con Octokit mockeado

ESLint · MCP Inspector · Antigravity IDE

09 Decisiones técnicas

Decisión	Por qué
stdio en vez de HTTP	Es un proceso local: no abre puertos y el token nunca sale de la máquina.
Errores como resultado con `isError`	No rompen el protocolo: el LLM los lee y se los explica al usuario.
Se rechazan los parámetros desconocidos	Si se ignoraran, el agente podría confirmar algo que no hizo.
Reglas propias para el nombre del repo	Solo letras, números y guiones, de 3 a 100 caracteres. Falla antes de llamar a GitHub, con un mensaje claro.
Node 22 en vez de Node 18	Node 18 ya no tiene soporte y las dependencias actuales no corren en él: ESLint 10 exige Node 22.13 y Octokit, Node 20.

10 tools

10

Principales

- `create_repository`
- `list_repositories`
- `create_issue`
- `list_issues`
- `create_commit`

Avanzados

- `create_branch`
- `create_pull_request`
- `list_commits`
- `close_issue`

Diagnóstico

- `health_check`

11 Validación con Zod

Pedido: “Crea un repositorio llamado `mi repo`”

→

`Parámetros inválidos. name: El nombre del repositorio solo admite letras, números y guiones.`

→

El server lo rechaza **sin llamar a GitHub** y el agente explica el error.

```
name: z.string()
  .min(3, "El nombre del repositorio debe tener al menos 3 caracteres.")
  .max(100, "El nombre del repositorio no puede superar los 100 caracteres.")
  .regex(/^[a-zA-Z0-9-]+$/, "El nombre del repositorio solo admite letras, números y guiones.")
```

12 Manejo de errores

Tipo	Cuándo	Mensaje
`ValidationError`	Input inválido	“El nombre del repositorio solo admite letras, números y guiones.”
`AuthenticationError`	401 / 403	“GitHub rechazó las credenciales. Verifica que GITHUB_TOKEN sea válido…”
`GitHubAPIError`	404, 422, rate limit	“El repositorio X no fue encontrado. Verifica el nombre e intenta de nuevo.”
`NetworkError`	Sin conexión	“No se pudo conectar con GitHub. Verifica tu conexión a internet…”

Rate limit: respeto el tiempo de espera que indica GitHub, hago hasta 3 intentos y, si la espera supera los 30 segundos, informo cuándo reintentar en lugar de bloquear al agente.

13 El código más interesante

Todos los tools pasan por la misma función: validar, ejecutar y traducir el error.

validar

ejecutar

traducir el error

```
run: async (rawArguments) ⇒ {
  try {
    const parsed = schema.safeParse(rawArguments ?? {});
    if (!parsed.success) throw fromZodError(parsed.error);

    const output = await execute(parsed.data);
    return { content: [{ type: "text", text: JSON.stringify(output) }] };
  } catch (error) {
    const appError = toAppError(error);
    return { content: [{ type: "text", text: appError.message }], isError: true };
  }
}
```

14 Seguridad del token

01

El token vive en `.env`, que está en `.gitignore` y nunca se subió.

02

Los logs van a stderr y ocultan cualquier valor con formato de token.

03

En Antigravity bloqueé la lectura de `.env` y los comandos

`node` `gh` `curl` `cat` `type` y `Get-Content` .

04

Se aplica el principio de mínimo privilegio: el agente solo actúa a través de los tools.

15

Testing

127

tests

con Vitest, en 7 archivos.

- Schemas: inputs válidos e inválidos, con sus mensajes.
- Operaciones con Octokit mockeado: repo inexistente, credenciales inválidas, repo vacío.
- Traducción de errores: 404, 401, 403, 422, rate limit y red.
- Reintentos con timers simulados.
- Logs sin el token.
- Flujo MCP completo con un cliente en memoria.

Demo en vivo

1. “Verifica la conexión con GitHub y lista mis 5 repositorios más recientes.”
2. “Crea un repositorio llamado `demo-defensa`, abre un issue ’Agregar documentación’ y muéstrame los issues abiertos.”
3. “En `demo-defensa`, crea la rama `feat/docs`, agrega `docs/guia.md` y abre un pull request hacia `main`.”
4. “Crea un repositorio llamado `mi repo`. Si el nombre no es válido, no lo cambies: decime cuál fue el error.” (caso de error)
5. En la terminal: `npm test` (tests pasando)

17 Desafíos y cómo los resolví

Desafío	Solución
Antigravity fallaba con todos los modelos (`429 RESOURCE_EXHAUSTED`)	→ Leí los logs: era un límite de Google, no del server. Se resolvió actualizando Antigravity
El agente leyó `.env` y usó la API por la terminal	→ Regeneré el token y bloqueé esos permisos del agente
Parámetros que el tool no admite se ignoraban sin avisar	→ Ahora se rechazan, con la lista de los que sí acepta
Gemini rechazaba partes del JSON Schema de los tools	→ Quité `$schema` y `additionalProperties` , y la validación sigue en Zod

18 Aprendizajes

01 La descripción de un tool es parte del código: de ella depende que el agente elija bien.

02 Los errores también son para el LLM: tienen que ser claros para que los pueda explicar.

03 Hay que revisar qué tool usó el agente, no solo su respuesta: una respuesta correcta puede venir por un camino equivocado.

04 Un agente con acceso al proyecto necesita límites explícitos.

Qué haría diferente: sumar transporte HTTP para usarlo de forma remota y tools para labels y comentarios en issues.

19 Uso de IA

Desarrollé el proyecto con **Claude Code** como asistente:

1. Leí la documentación y pedí un plan técnico antes de escribir código.
2. Aprobé cada decisión que la consigna no definía.
3. Probé yo los 10 tools en Antigravity.
4. Pedí una auditoría final contra la rúbrica.

¡Gracias!

¿Preguntas?

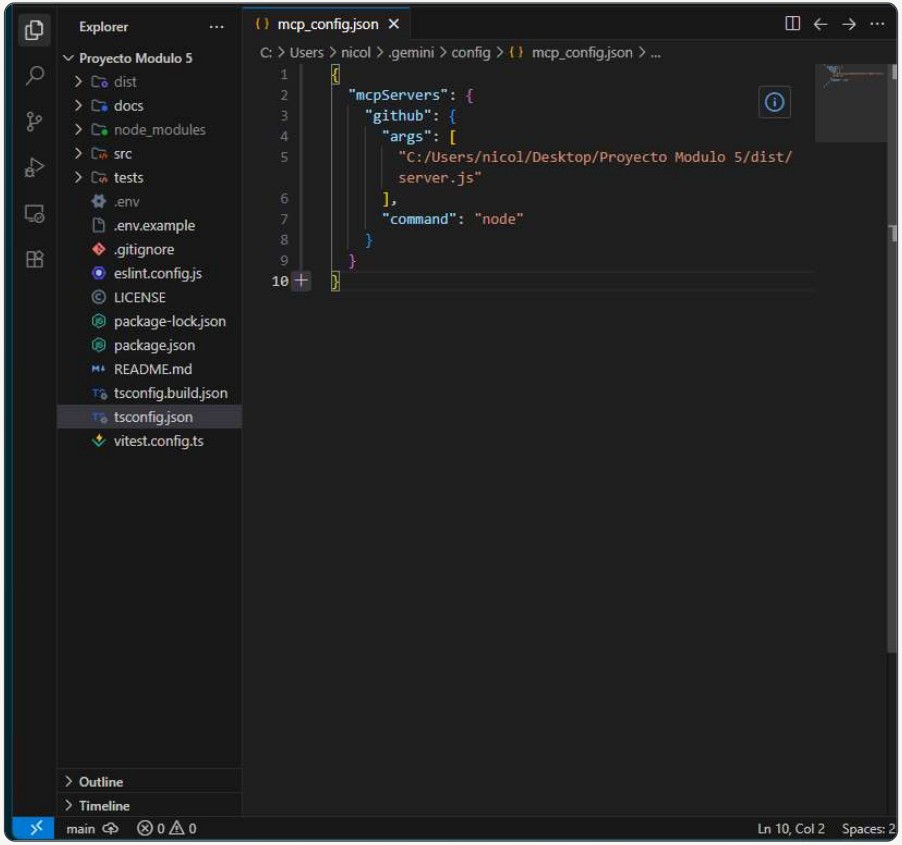
---

github.com/rominababiel/github-mcp-server

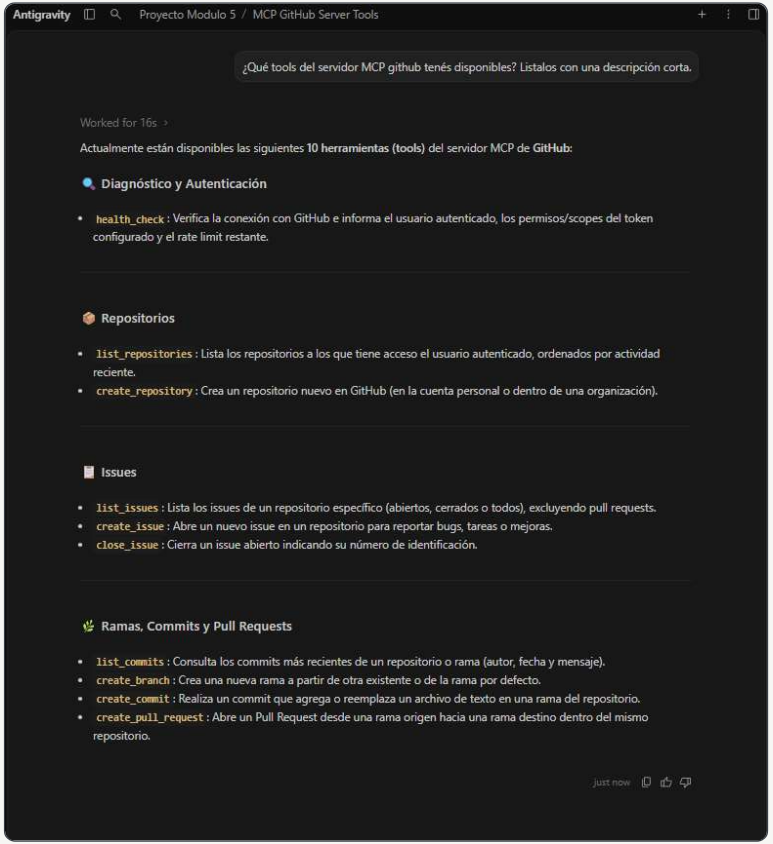
Plan B

Arquitectura

Configuración del server en Antigravity: lo lanza con node como proceso local.



Plan B **10 tools** El agente lista los 10 tools del server con su descripción.



Plan B

Demo · paso 1

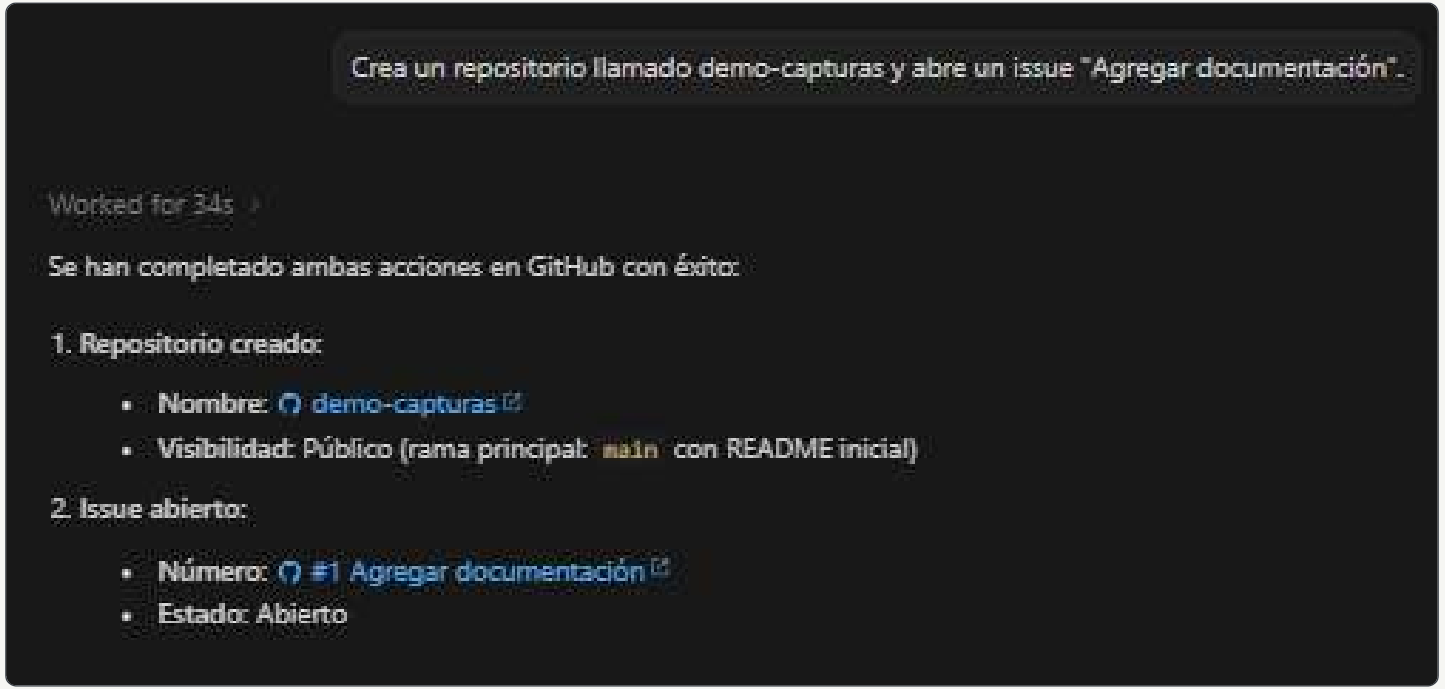
health_check: conexión verificada, usuario, permisos y rate limit.



Plan B

Demo · paso 2

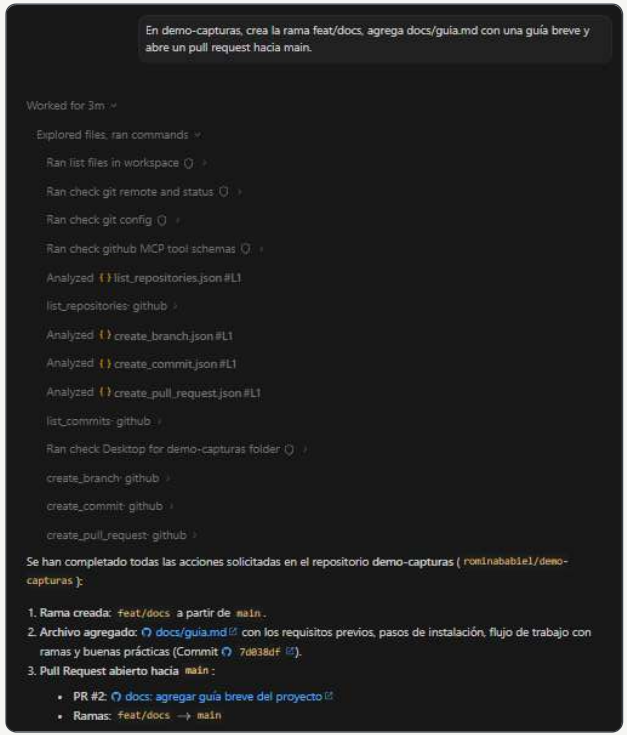
Crea el repositorio y abre el issue.

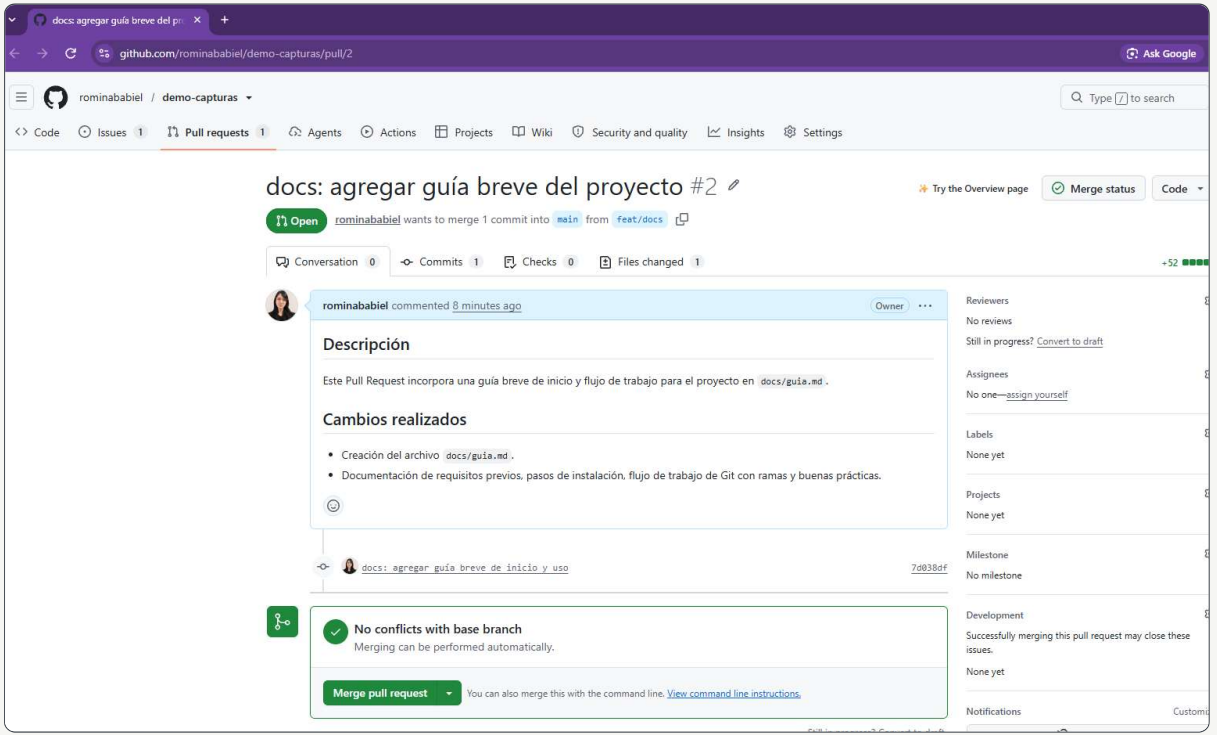


Plan B

Demo · paso 3

Rama, commit y pull request, y el PR abierto en GitHub.

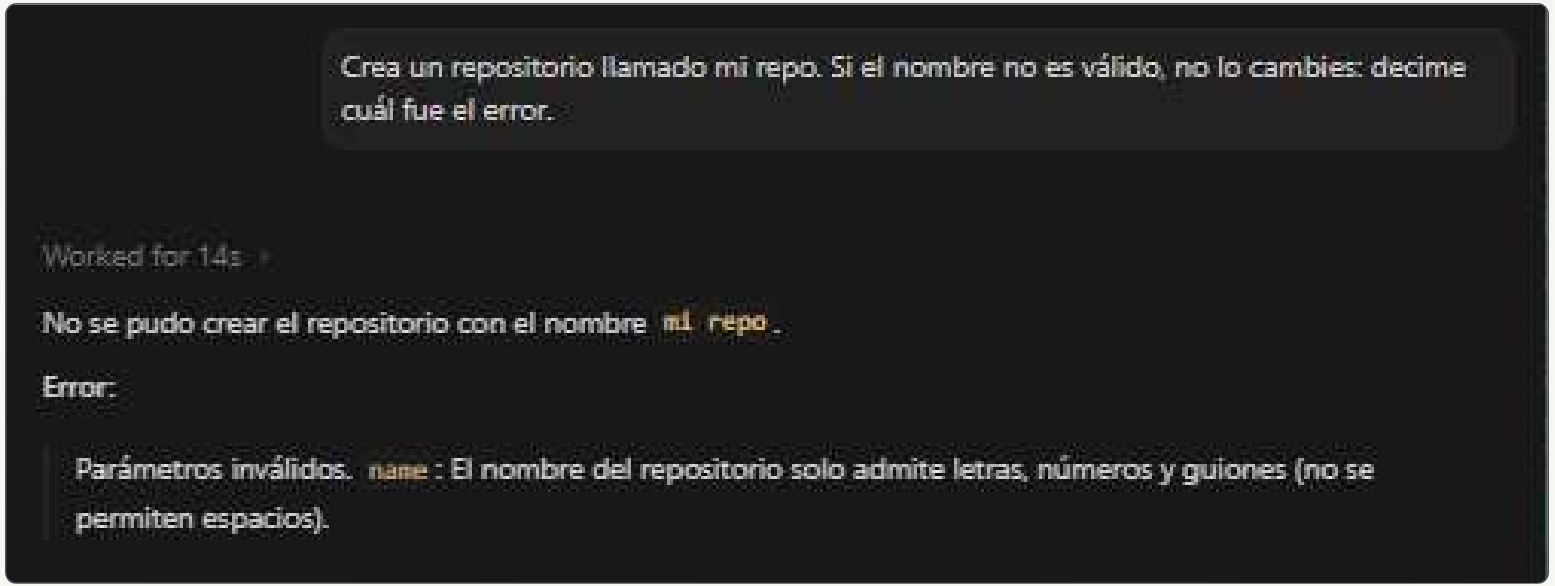




Plan B

Demo · paso 4

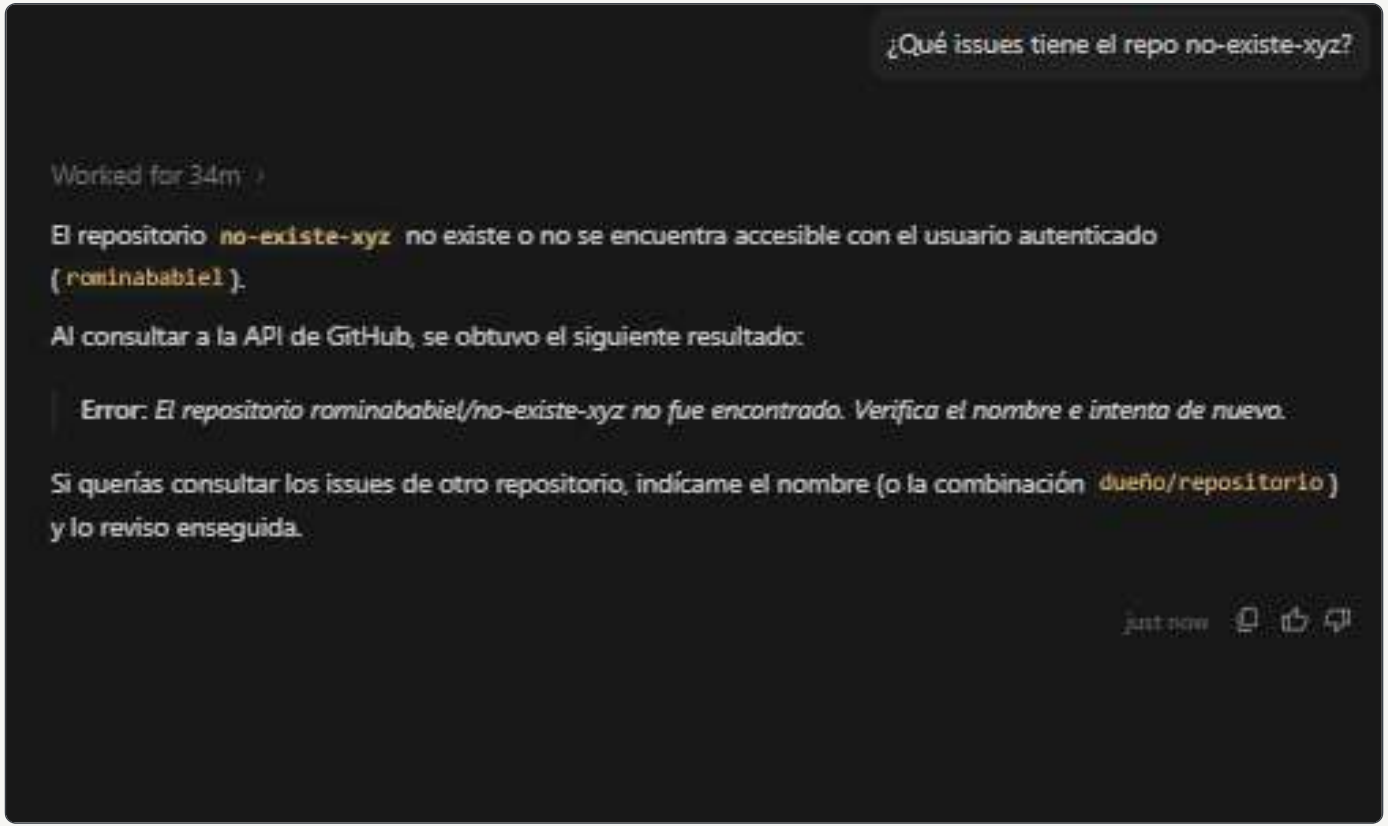
Caso de error: el server rechaza “mi repo” y el agente explica por qué.



Plan B

Manejo de errores

Repo inexistente: el 404 vuelve traducido a lenguaje natural.



Plan B

Seguridad del token

Comandos bloqueados para el agente en Antigravity.

